```
template<typename T>
class queue
{
private:
    struct node
    {
        std::shared_ptr<T> data;     <-1
        std::unique_ptr<node> next;
    };

    std::unique_ptr<node> head;
    node* tail;

public:
    queue():
        head(new node),tail(head.get())    <-2
    {}

    queue(const queue& other)=delete;
    queue& operator=(const queue& other)=delete;

    std::shared_ptr<T> try_pop()
    {

        if(head.get()==tail)       <-3
        {
            return std::shared_ptr<T>();
        }
        std::shared_ptr<T> const res(head->data);        <-4
        std::unique_ptr<node> old_head=std::move(head);
        head=std::move(old_head->next);                  <-5
        return res;                      <-6
    }

    void push(T new_value)
    {
        std::shared_ptr<T> new_data(
            std::make_shared<T>(std::move(new_value)));      <-7
        std::unique_ptr<node> p(new node);               <-8
        tail->data=new_data;                 <-9
        node* const new_tail=p.get();
        tail->next=std::move(p);
        tail=new_tail;
    }
};
```

The changes to `try_pop()` are fairly minimal. First, you're comparing `head` against `tail` ❸ rather than checking for `NULL`, because the dummy